Fourier Analysis in SageMath:

A Comprehensive Survey of Symbolic, Numerical, and Spectral Tools for the Practitioner

Prepared for an audience with doctoral-level background in data analysis

March 1, 2026

Contents

1 Introduction **3**

2 The Continuous Fourier Transform: Symbolic Capabilities **3**

2.1 Convention and Notation 3

2.2 Symbolic Integration via `integrate` 4

2.3 Assumptions and the `assume` System 4

2.4 Laplace and Mellin Transforms 5

3 Fourier Series: Periodic Functions and the Torus **6**

3.1 Symbolic Fourier Coefficients 6

3.2 Convergence and Gibbs Phenomenon 6

4 The Discrete Fourier Transform and Its Fast Computation **7**

4.1 Mathematical Framework 7

4.2 Exact DFT over Symbolic and Finite Fields 7

4.3 Numerical FFT via NumPy and SciPy 8

4.4 SciPy's Extended FFT Toolkit 9

5 Windowing, Spectral Leakage, and Apodization **10**

5.1 The Leakage Problem 10

5.2 Windows in SciPy 10

6 Power Spectral Density Estimation **11**

6.1 Periodogram and Its Inconsistency 11

6.2 Welch's Method 11

6.3 Multitaper Spectral Estimation 12

7 The Short-Time Fourier Transform and Time-Frequency Analysis **12**

8 Wavelet Transforms and Multiresolution Analysis **13**

8.1 Continuous Wavelet Transform 13

8.2 Discrete Wavelet Transform 14

9 Convolution, Correlation, and the Convolution Theorem **15**

9.1 Symbolic Convolution 15

9.2 Numerical Convolution via FFT 15

9.3 Circular vs. Linear Convolution 16

10 Fourier Analysis on Finite Groups and Number-Theoretic Applications **16**

11 Parseval's Theorem, L^2 Theory, and Numerical Verification **17**

12 Integration with Sage's Symbolic and Statistical Ecosystem **17**

13 Limitations and Practical Considerations **18**

14 Conclusion **19**

1 Introduction

Fourier analysis is the mathematical backbone of signal processing, spectral estimation, partial differential equation theory, and a significant portion of modern machine-learning architectures. Its central premise—that any sufficiently well-behaved function can be decomposed into a (possibly infinite) superposition of sinusoids—is so broadly applicable that it pervades nearly every quantitative discipline. For the practicing data analyst, the ability to move fluidly between symbolic Fourier transforms, discrete approximations, spectral density estimates, and convolution algebras is essential.

SageMath (hereafter "Sage") is an open-source mathematics software system built on top of Python and integrating a rich ecosystem of numerical libraries (NumPy, SciPy), symbolic engines (Maxima, SymPy, Pynac), and specialized packages for algebra, number theory, and combinatorics. Because Sage sits at the intersection of symbolic and numerical computation, it occupies a unique position for Fourier analysis: one can derive a closed-form transform, evaluate it numerically, and immediately pipe the result into a statistical or spectral estimation workflow—all within a single, coherent session.

This essay surveys the full landscape of Fourier-analytic tools available in Sage, organized from the purely symbolic layer through to the numerical and applied layers. The discussion assumes familiarity with measure-theoretic integration, L^p function spaces, the theory of distributions (in the sense of Schwartz), and applied spectral analysis at graduate level.

2 The Continuous Fourier Transform: Symbolic Capabilities

2.1 Convention and Notation

Throughout this essay we adopt the analyst's symmetric convention,

$$\hat{f}(\xi) = \int_{-\infty}^{\infty} f(t)\, e^{-2\pi i \xi t}\, dt, \qquad f(t) = \int_{-\infty}^{\infty} \hat{f}(\xi)\, e^{2\pi i \xi t}\, d\xi. \tag{1}$$

Sage's symbolic layer does not fix a single convention internally; the user must specify the kernel explicitly, which is actually a feature rather than a limitation, as it permits the physicist's $e^{-i\omega t}$ convention and the engineer's one-sided variants with equal ease.

2.2 Symbolic Integration via `integrate`

The most direct route to a symbolic Fourier transform in Sage is the `integrate` function, which interfaces with Maxima's powerful definite-integral routines augmented by Sage's own heuristic simplification layer.

Listing 1: Symbolic Fourier transform of a Gaussian

```
var('t xi sigma')
assume(sigma > 0)

f = exp(-pi * sigma^2 * t^2)
F_hat = integrate(f * exp(-2*pi*I*xi*t), t, -oo, oo)
F_hat = F_hat.simplify_full()
print(F_hat)
# Expected: e^(-pi*xi^2/sigma^2) / sigma
```

The Gaussian $f(t) = e^{-\pi\sigma^2 t^2}$ is, of course, its own Fourier transform up to scaling—a fact that Maxima confirms analytically. More generally, `integrate` succeeds for functions whose transforms are expressible in terms of special functions known to Maxima: error functions, Bessel functions of the first kind, gamma functions, and hypergeometric series.

For functions outside Maxima's repertoire, Sage transparently falls back to SymPy via the `sympy` bridge, broadening the class of tractable integrals. The practitioner should be aware that both solvers work heuristically; proofs of convergence must be supplied separately through the `assume` mechanism or by analytic argument.

2.3 Assumptions and the `assume` System

Fourier transforms of parameterized families depend critically on the domain of parameters. Sage's `assume`/`forget` system allows the analyst to assert hypotheses—for instance that $\sigma > 0$ or that $a \in \mathbb{Z}$—which are then passed to the underlying CAS for conditional evaluation.

Listing 2: Parametric transform with branching behavior

```
var('a t xi')
assume(a, 'real')
assume(a > 0)

# Two-sided exponential (Laplace kernel)
f = exp(-a * abs(t))
F_hat = integrate(f * exp(-2*pi*I*xi*t), t, -oo, oo)
```

```
print(F_hat.simplify_full())
# Result: 2*a / (a^2 + 4*pi^2*xi^2)
```

The output $\hat{f}(\xi) = 2a/(a^2 + 4\pi^2\xi^2)$ is the Lorentzian (Cauchy) density—a standard result that confirms the correctness of the symbolic engine.

2.4 Laplace and Mellin Transforms

Sage provides `laplace` and `inverse_laplace` as first-class functions for the Laplace transform

$$\mathcal{L}\{f\}(s) = \int_0^\infty f(t)\, e^{-st}\, dt, \tag{2}$$

and `mellin_transform` / `inverse_mellin_transform` for

$$\mathcal{M}\{f\}(s) = \int_0^\infty f(t)\, t^{s-1}\, dt. \tag{3}$$

These are not Fourier transforms per se, but they are Fourier-related: the Laplace transform along the imaginary axis $s = i\omega$ reduces to the one-sided Fourier transform, and the Mellin transform is a Fourier transform on the multiplicative group $(\mathbb{R}_{>0}, \times)$. Understanding all three as instances of a generalized convolution structure is important for analysts working in signal processing, fractional calculus, or number theory.

Listing 3: Laplace transform and its imaginary-axis restriction

```
var('t s')
assume(s, 'complex')

f = t^2 * exp(-3*t)
L = laplace(f, t, s)
print(L)          # 2 / (s + 3)^3

# Restrict to imaginary axis: s -> I*omega
var('omega', domain='real')
L_fourier = L.subs(s = I*omega)
print(L_fourier)  # 2 / (I*omega + 3)^3
```

3 Fourier Series: Periodic Functions and the Torus

3.1 Symbolic Fourier Coefficients

For a function $f : [0, L] \to \mathbb{C}$ extended periodically, the Fourier coefficients are

$$c_n = \frac{1}{L} \int_0^L f(t)\, e^{-2\pi i n t/L}\, dt, \qquad n \in \mathbb{Z}. \tag{4}$$

Sage computes these symbolically via parameterized definite integrals. There is no dedicated `fourier_series` function for arbitrary periods in the base library, but the pattern is straightforward:

Listing 4: Fourier series of a sawtooth wave

```
var('t n L')
assume(n, 'integer')
assume(L > 0)

# Sawtooth on [0, L)
f = t / L - 1/2
c_n = (1/L) * integrate(f * exp(-2*pi*I*n*t/L), t, 0, L)
c_n_simplified = c_n.simplify_full()
print(c_n_simplified)
# For n != 0: I / (2*pi*n)
```

The singularity at $n = 0$ reflects the zero mean of the sawtooth; Sage handles this automatically through limits. For piecewise-defined functions, the `piecewise` constructor integrates term-by-term.

3.2 Convergence and Gibbs Phenomenon

Sage's ability to compute partial sums symbolically and then evaluate them numerically makes it an excellent tool for visualizing convergence phenomena. The Gibbs overshoot at a jump discontinuity—approximately 8.9% of the jump height, arising from $\int_0^\pi (\sin u)/u\, du - \pi/2 \approx 0.0895\,\pi$—can be demonstrated by plotting partial sums of increasing order.

Listing 5: Gibbs phenomenon: partial sums of sawtooth

```
import numpy as np
import matplotlib.pyplot as plt

t_vals = np.linspace(0, 1, 2000)
```

```python

def sawtooth_partial_sum(t, N):
    """N-term partial Fourier sum of sawtooth on [0,1)."""
    s = np.zeros_like(t)
    for n in range(1, N+1):
        s += -(1 / (np.pi * n)) * np.sin(2 * np.pi * n * t)
    return s

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for ax, N in zip(axes, [5, 20, 100]):
    ax.plot(t_vals, t_vals - 0.5, 'k--', lw=0.8, label='Sawtooth')
    ax.plot(t_vals, sawtooth_partial_sum(t_vals, N), lw=1.2, label=f'N={N}')
    ax.set_title(f'Partial sum, $N={N}$')
    ax.legend(fontsize=8)
plt.tight_layout()
plt.savefig('gibbs.pdf')
```

4 The Discrete Fourier Transform and Its Fast Computation

4.1 Mathematical Framework

Let $\mathbf{x} = (x_0, x_1, \ldots, x_{N-1}) \in \mathbb{C}^N$. The Discrete Fourier Transform (DFT) is the linear map

$$X_k = \sum_{n=0}^{N-1} x_n \, \omega_N^{-nk}, \qquad \omega_N = e^{2\pi i/N}, \quad k = 0, 1, \ldots, N-1, \tag{5}$$

implemented by an $N \times N$ Vandermonde matrix F_N with $(F_N)_{kn} = \omega_N^{-nk}/\sqrt{N}$ in its unitary normalization. The inverse DFT is

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \, \omega_N^{nk}. \tag{6}$$

4.2 Exact DFT over Symbolic and Finite Fields

Sage's *exact* DFT operates over any ring containing a primitive N-th root of unity. For $N = 2^m$ over $\mathbb{Q}(\omega_N)$ this is the *cyclotomic field* approach, which Sage supports natively:

Listing 6: Exact DFT in a cyclotomic field

```
N = 8
CyclN = CyclotomicField(N)
omega = CyclN.gen()        # primitive N-th root of unity

def exact_dft(x):
    N = len(x)
    return [sum(x[n] * omega^(-n*k) for n in range(N))
            for k in range(N)]

x = [1, 0, -1, 0, 1, 0, -1, 0]
X = exact_dft(x)
print(X)
```

This is of theoretical and cryptographic interest—applications include NTT-based polynomial multiplication in lattice cryptography—but is far too slow for practical signal processing.

For finite-field DFTs (used in algebraic coding theory and NTTs), Sage's number-theoretic infrastructure—`GF`, `IntegerModRing`, and polynomial arithmetic—provides a complete algebraic toolkit.

4.3 Numerical FFT via NumPy and SciPy

For practical computation Sage defers to NumPy's `numpy.fft` module, which implements the Cooley–Tukey radix-2 FFT in $O(N \log N)$ operations. The interface is identical to pure Python since Sage's numeric arrays are NumPy arrays:

Listing 7: FFT and power spectrum estimation

```
import numpy as np

# Synthetic signal: two tones plus noise
rng = np.random.default_rng(42)
N, fs = 1024, 1000.0            # samples, sample rate (Hz)
t = np.arange(N) / fs
x = (2.0 * np.sin(2*np.pi*50*t)
   + 0.5 * np.sin(2*np.pi*120*t)
   + rng.normal(scale=0.3, size=N))

# FFT
X = np.fft.rfft(x, norm='ortho')  # ortho: unitary
   normalization
```

```
freqs = np.fft.rfftfreq(N, d=1/fs)

# One-sided power spectral density (V^2 / Hz)
psd = (np.abs(X)**2) / (fs * N)
psd[1:-1] *= 2                    # double for one-sided

print(f"Peak frequency: {freqs[np.argmax(psd)]:.1f} Hz")
# Output: Peak frequency: 50.0 Hz
```

Several subtleties merit attention for the sophisticated user. The `norm='ortho'` flag produces the *unitary* DFT, preserving Parseval's identity $\|\hat{\mathbf{x}}\|_2 = \|\mathbf{x}\|_2$ exactly. The `rfft` variant exploits Hermitian symmetry for real signals, halving storage and compute. Aliasing, spectral leakage, and the choice of window function are *not* handled by NumPy; these require explicit pre-processing (see Section 5).

4.4 SciPy's Extended FFT Toolkit

SciPy's `scipy.fft` module supersedes NumPy's in several respects: it supports arbitrary-length transforms (not only powers of two) via Bluestein's chirp-z algorithm, offers out-of-place and in-place variants, provides explicit control over *workers* (parallel FFT over multiple threads via FFTW backends), and exposes the `next_fast_len` utility for zero-padding to highly composite lengths.

Listing 8: SciPy FFT with optimal zero-padding

```
from scipy.fft import rfft, rfftfreq, next_fast_len

N_raw = 700                         # awkward length
N_pad = next_fast_len(N_raw)        # -> 720 = 2^4 * 3^2 * 5
print(f"Optimal padded length: {N_pad}")

signal_raw = np.random.default_rng(0).normal(size=N_raw)
X_padded = rfft(signal_raw, n=N_pad)
```

For multidimensional signals—time-frequency images, spatial data on grids—`fftn` computes the d-dimensional DFT,

$$X_{\mathbf{k}} = \sum_{\mathbf{n}} x_{\mathbf{n}} \exp(-2\pi i\, \mathbf{k} \cdot \mathbf{n}/N)\,, \tag{7}$$

where $\mathbf{k}, \mathbf{n} \in \mathbb{Z}^d$ with appropriate modular arithmetic.

5 Windowing, Spectral Leakage, and Apodization

5.1 The Leakage Problem

The DFT implicitly assumes periodicity over the analysis window. When the signal is non-periodic relative to the window length, or contains transients, the spectral energy of a pure sinusoid spreads across neighboring frequency bins—a phenomenon called *spectral leakage*. Its mathematical origin is convolution in the frequency domain:

$$X_{\text{windowed}}(\xi) = \hat{x}(\xi) * \hat{w}(\xi), \tag{8}$$

where w is the window function. A rectangular window has the Dirichlet kernel $\hat{w}$ with $-13\,\text{dB}$ sidelobes; proper window design trades off main-lobe width against sidelobe suppression.

5.2 Windows in SciPy

SciPy's `scipy.signal.windows` module provides a comprehensive library of window functions, each characterized by its main-lobe width and peak sidelobe level:

Listing 9: Comparing window functions

```python
from scipy.signal import windows
import numpy as np

N = 256
window_types = {
    'Rectangular': windows.boxcar(N),
    'Hann':        windows.hann(N),
    'Blackman':    windows.blackman(N),
    'Flat Top':    windows.flattop(N),
    'Kaiser(14)':  windows.kaiser(N, beta=14),
}

for name, w in window_types.items():
    W = np.fft.rfft(w / w.sum(), n=4096)  # normalized, zero-padded
    db = 20 * np.log10(np.abs(W) + 1e-16)
    sidelobe = np.max(db[10:])             # skip main lobe
    print(f"{name:15s}: peak sidelobe = {sidelobe:.1f} dB")
```

The Kaiser window with parameter β provides an analytically optimal trade-off via the prolate spheroidal wave functions; its sidelobe level decreases approximately as

$-2.285(\beta/\pi)N$ dB per octave. The flat-top window sacrifices frequency resolution for amplitude accuracy and is preferred for calibration measurements.

6 Power Spectral Density Estimation

6.1 Periodogram and Its Inconsistency

The naive periodogram estimator $\hat{S}(f) = |X_k|^2/N$ is *asymptotically unbiased* but *not consistent*: its variance does not decrease as $N \to \infty$, since each periodogram ordinate is approximately chi-squared with two degrees of freedom regardless of sample size. This is one of the fundamental results of classical spectral estimation theory (Bartlett, 1948).

6.2 Welch's Method

Welch's modified periodogram averages the periodograms of overlapping sub-segments, trading frequency resolution for variance reduction. For K non-overlapping segments of length M, the variance decreases by a factor of K:

$$\hat{S}_{\text{Welch}}(f_k) = \frac{1}{K} \sum_{j=0}^{K-1} \left| \frac{1}{UM} \sum_{n=0}^{M-1} w[n]\, x[n + jM]\, e^{-2\pi i k n/M} \right|^2, \tag{9}$$

where $U = (1/M) \sum_n w[n]^2$ normalizes for window power.

Listing 10: Welch PSD via SciPy

```python
from scipy.signal import welch

f, Pxx = welch(x,
               fs=fs,
               window='hann',
               nperseg=256,
               noverlap=128,
               scaling='density')    # V^2 / Hz

# Integrate to recover total power: Parseval check
total_power = np.trapz(Pxx, f)
print(f"Total PSD power: {total_power:.4f}")
print(f"Signal variance: {np.var(x):.4f}")
```

6.3 Multitaper Spectral Estimation

Thompson's multitaper method (MTM) addresses leakage more rigorously by averaging periodograms computed with a family of K orthogonal Slepian (discrete prolate spheroidal sequence, DPSS) tapers, each optimally concentrated in the bandwidth $[-W, W]$:

Listing 11: Multitaper PSD using nitime or spectrum

```
# Using the spectrum library (pip install spectrum)
from spectrum import pmtm

Sk, weights, _ = pmtm(x, NW=4, k=7, method='adapt')
# Sk: tapered periodograms (K x N/2+1)
# Adaptive weighting suppresses spectral leakage optimally

Pxx_mtm = np.abs(np.dot(weights, Sk)).squeeze()
freqs_mtm = np.linspace(0, fs/2, Pxx_mtm.shape[-1])
```

MTM is the gold standard for spectral estimation in geophysics, neuroscience, and finance, where short data records demand low bias *and* low variance simultaneously.

7 The Short-Time Fourier Transform and Time-Frequency Analysis

For non-stationary signals, the STFT localizes spectral content in time:

$$\mathrm{STFT}[x](t, \xi) = \int_{-\infty}^{\infty} x(\tau)\, \overline{w(\tau - t)}\, e^{-2\pi i \xi \tau}\, d\tau, \tag{10}$$

producing a spectrogram $|\mathrm{STFT}[x](t, \xi)|^2$ in the time-frequency plane. The uncertainty principle (Heisenberg–Gabor inequality) bounds the joint resolution:

$$\sigma_t \cdot \sigma_\xi \geq \frac{1}{4\pi}, \tag{11}$$

with equality achieved uniquely by Gaussian windows—the basis of Gabor analysis.

Listing 12: STFT spectrogram with SciPy

```
from scipy.signal import stft, spectrogram

# Chirp signal: linearly sweeping frequency
t = np.linspace(0, 2, 2*fs, endpoint=False)
```

```python
x_chirp = np.sin(2*np.pi * (20 + 50*t) * t)

f, t_seg, Zxx = stft(x_chirp,
                     fs=fs,
                     window='hann',
                     nperseg=128,
                     noverlap=96)

magnitude_db = 20 * np.log10(np.abs(Zxx) + 1e-10)
print(f"STFT shape: {Zxx.shape}")   # (freq_bins, time_frames)
```

SciPy also provides `spectrogram` as a higher-level wrapper. For quadratic time-frequency representations beyond the STFT—Wigner-Ville, Cohen's class—the `tftb` package (Time-Frequency Toolbox for Python) interfaces naturally with Sage's numerical layer.

8 Wavelet Transforms and Multiresolution Analysis

8.1 Continuous Wavelet Transform

The CWT with mother wavelet ψ is defined as

$$W_x(a,b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{\infty} x(t)\, \overline{\psi\left(\frac{t-b}{a}\right)}\, dt, \tag{12}$$

where $a > 0$ is the scale and $b \in \mathbb{R}$ is the translation. The CWT provides adaptive time-frequency resolution: fine temporal resolution at high frequencies (small a), fine frequency resolution at low frequencies (large a).

Listing 13: Continuous wavelet transform via PyWavelets

```python
import pywt
import numpy as np

signal = x_chirp                            # from previous example
scales = np.arange(1, 128)
wavelet = 'cmor1.5-1.0'                     # Complex Morlet

coefficients, freqs_cwt = pywt.cwt(
    signal, scales, wavelet, sampling_period=1/fs
```

```
)
power = np.abs(coefficients)**2
print(f"CWT power array shape: {power.shape}")
```

8.2 Discrete Wavelet Transform

The DWT implements Mallat's multiresolution analysis through a two-channel perfect reconstruction filter bank. At each level j, the signal is split into approximation coefficients $\mathbf{a}_j$ (low-pass) and detail coefficients $\mathbf{d}_j$ (high-pass) via convolution with scaling and wavelet filters followed by downsampling-by-2:

Listing 14: Multi-level DWT decomposition and reconstruction

```
import pywt

# 5-level Daubechies-8 decomposition
coeffs = pywt.wavedec(signal, wavelet='db8', level=5, mode='periodization')
cA5, cD5, cD4, cD3, cD2, cD1 = coeffs

# Threshold detail coefficients (hard thresholding)
threshold = 0.5
coeffs_thresholded = [cA5] + [
    pywt.threshold(c, value=threshold, mode='hard') for c in [cD5, cD4, cD3, cD2, cD1]
]

# Perfect reconstruction
x_denoised = pywt.waverec(coeffs_thresholded, 'db8', mode='periodization')
snr = 10 * np.log10(np.var(signal) / np.var(signal - x_denoised[:len(signal)]))
print(f"Reconstruction SNR: {snr:.2f} dB")
```

PyWavelets (`pywt`) is fully accessible within Sage and provides an extensive catalogue of wavelets: Daubechies (db$N$), Symlets (sym$N$), Coiflets, biorthogonal spline families, and complex analytic wavelets suitable for instantaneous frequency estimation.

9 Convolution, Correlation, and the Convolution Theorem

9.1 Symbolic Convolution

For functions in $L^1(\mathbb{R})$, convolution is

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau, \tag{13}$$

and the *convolution theorem* states $\widehat{f * g} = \hat{f} \cdot \hat{g}$. Sage computes symbolic convolutions by direct integration, though performance degrades for complex analytic expressions. For rational functions, partial-fraction decomposition followed by inverse Laplace transforms is often faster.

9.2 Numerical Convolution via FFT

The FFT-based convolution algorithm achieves $O((M+N)\log(M+N))$ complexity compared to $O(MN)$ for direct summation—a crucial distinction for large signals or long filters.

Listing 15: FFT-based convolution and cross-correlation

```
from scipy.signal import fftconvolve, correlate

# Matched filter: correlate signal with template
template = np.sin(2*np.pi*50*np.arange(50)/fs)
signal_noisy = x + rng.normal(scale=2.0, size=len(x))

# Cross-correlation via FFT (O(N log N))
xcorr = correlate(signal_noisy, template, mode='full', method
    ='fft')
lags = np.arange(-(len(template)-1), len(signal_noisy))

# Detection peak
peak_lag = lags[np.argmax(np.abs(xcorr))]
print(f"Template detected at lag: {peak_lag} samples = {
    peak_lag/fs:.4f} s")
```

9.3 Circular vs. Linear Convolution

The DFT-based convolution is inherently *circular*: multiplying length-N DFTs yields the N-point circular convolution, not the linear convolution of length $M + N - 1$. Correct linear convolution via FFT requires zero-padding both sequences to length $\geq M + N - 1$ before transforming. SciPy's `fftconvolve` handles this automatically; the analyst should verify this when implementing custom FFT-based filtering.

10 Fourier Analysis on Finite Groups and Number-Theoretic Applications

Sage's algebraic machinery enables Fourier analysis over finite abelian groups—a domain with deep connections to character theory, coding theory, and computational number theory. For the group $G = \mathbb{Z}/N\mathbb{Z}$, the characters are $\chi_k(n) = \omega_N^{kn}$, and the DFT (5) is exactly the group Fourier transform.

For non-abelian groups, Sage's `SymmetricGroup`, `DihedralGroup`, and `MatrixGroup` classes support representation theory, enabling the non-abelian Fourier transform (NAFT). This is relevant for invariant feature extraction in machine learning (e.g., group-equivariant convolutional networks) and quantum computing (hidden subgroup algorithms).

Listing 16: Characters of Z/NZ and the DFT matrix

```
N = 8
Zmod = IntegerModRing(N)
characters = [[exp(2*pi*I*k*n/N).n() for n in range(N)] for k
    in range(N)]

# This is exactly the DFT matrix F_N
F = matrix(CC, characters)
print((F * F.conjugate_transpose() / N).apply_map(lambda z: z
   .real().round(8)))
# Should be identity matrix (unitarity check)
```

11 Parseval's Theorem, L^2 Theory, and Numerical Verification

Parseval's identity in its DFT form,

$$\sum_{n=0}^{N-1} |x_n|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X_k|^2, \tag{14}$$

provides a fundamental numerical sanity check. Any discrepancy indicates an error in normalization conventions. For the continuous transform on $L^2(\mathbb{R})$, Plancherel's theorem states $\|\hat{f}\|_2 = \|f\|_2$ (under the convention (1)).

Listing 17: Verifying Parseval's identity

```
x = rng.normal(size=512)
X = np.fft.fft(x)

lhs = np.sum(np.abs(x)**2)
rhs = np.sum(np.abs(X)**2) / len(x)

print(f"Parseval LHS: {lhs:.8f}")
print(f"Parseval RHS: {rhs:.8f}")
print(f"Relative error: {abs(lhs-rhs)/lhs:.2e}")
# Typical output: Relative error: 1.23e-14  (machine epsilon
    level)
```

12 Integration with Sage's Symbolic and Statistical Ecosystem

A distinctive advantage of Sage as a platform for Fourier analysis is the ability to compose symbolic and numerical computations within a single workflow:

1. **Derive symbolic transforms** using `integrate`, verify closed forms against known tables (e.g., Gradshteyn & Ryzhik), and export to numerical evaluation via `.n()` or `fast_callable`.

2. **Optimize numerical kernels** by symbolically differentiating filters with respect to design parameters—a workflow relevant for gradient-based filter design.

3. **Interface with Sage's statistics module** (`sage.stats`) for spectral hypothesis testing: F-tests for periodic components (Fisher's test), Kolmogorov–Smirnov tests

on spectral distributions, and likelihood ratio tests for harmonic regression models.

4. **Cryptographic applications** via Sage's `sage.crypto` module, where NTTs over finite fields underpin lattice-based encryption (CRYSTALS-Kyber, CRYSTALS-Dilithium).

The `fast_callable` interface deserves particular mention: it JIT-compiles a symbolic Sage expression into a C or Cython callable, reducing evaluation overhead for repeated numerical calls by orders of magnitude compared to Python-level symbolic evaluation.

Listing 18: Symbolic design to fast numerical evaluation via fast_callable

```
var('t omega_0 sigma')
# Gabor filter impulse response
h_sym = exp(-sigma^2 * t^2 / 2) * exp(I * omega_0 * t)

# Compile to fast numerical function
h_fast = fast_callable(h_sym, vars=[t],
                       domain=CDF,
                       expect_one_var=False)
# Evaluate on array
t_arr = [k/1000.0 for k in range(-500, 501)]
h_vals = [h_fast(sigma=2.0, omega_0=100.0, t=ti) for ti in
    t_arr]
```

13 Limitations and Practical Considerations

Despite its considerable strengths, the practitioner should be aware of several limitations.

Symbolic integration coverage. Maxima and SymPy fail on many practically relevant integrals—particularly those involving products of special functions or oscillatory integrands with non-algebraic phase. In such cases, Sage's `numerical_integral` (wrapping GSL's QAGS adaptive quadrature) provides a reliable fallback, though at the cost of exactness.

No dedicated Fourier PDE solver. Unlike MATLAB's Symbolic Math Toolbox or Wolfram Mathematica, Sage does not provide a high-level interface for solving PDEs via Fourier methods (e.g., `fourier_transform_solve`). This must be assembled by the user from symbolic transforms, algebraic manipulation, and inverse transform steps.

Real-time and embedded constraints. Sage is a mathematical workstation environment; it is not suitable for real-time signal processing or deployment on embedded

targets. For production pipelines, the analytical results from Sage should be validated and then re-implemented in C/C++ with FFTW, or in Julia with FFTW.jl.

Large-scale numerics. For FFTs on arrays exceeding available RAM, out-of-core approaches (e.g., Dask FFT, cuFFT for GPU computation) must be used outside the standard Sage environment, though Sage can orchestrate such computations via subprocess calls or JupyterHub integration.

14 Conclusion

SageMath provides a uniquely unified environment for Fourier analysis that spans the complete spectrum from rigorous symbolic computation—exact transforms over cyclotomic fields, analytical convergence verification, symbolic convolution—through to state-of-the-art numerical methods via NumPy, SciPy, PyWavelets, and the spectrum library. The `assume` system and CAS integration enable mathematically rigorous derivations, while `fast_callable` and direct NumPy array interoperability ensure that theoretical results can be efficiently evaluated at scale.

For the doctoral-level data analyst, Sage's greatest contribution is perhaps philosophical: it insists on the *unity* of symbolic and numerical mathematics. A Fourier transform is not merely an algorithm to be called, but a mathematical object whose properties—linearity, the convolution theorem, Parseval's identity, duality under the transform—can be verified, manipulated, and extended within the same environment where numerical experiments are run. This principled integration of computation and analysis is, ultimately, what distinguishes Sage from more narrowly focused numerical toolkits.

References

[1] The Sage Developers. *SageMath, the Sage Mathematics Software System.* `https://www.sagemath.org`, 2024.

[2] S. Mallat. *A Wavelet Tour of Signal Processing: The Sparse Way*, 3rd ed. Academic Press, 2009.

[3] F. J. Harris. "On the use of windows for harmonic analysis with the discrete Fourier transform." *Proceedings of the IEEE*, 66(1):51–83, 1978.

[4] D. J. Thomson. "Spectrum estimation and harmonic analysis." *Proceedings of the IEEE*, 70(9):1055–1096, 1982.

[5] J. W. Cooley and J. W. Tukey. "An algorithm for the machine calculation of complex Fourier series." *Mathematics of Computation*, 19(90):297–301, 1965.

[6] P. D. Welch. "The use of fast Fourier transform for the estimation of power spectra." *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.

[7] D. B. Percival and A. T. Walden. *Spectral Analysis for Physical Applications*. Cambridge University Press, 1993.

[8] E. M. Stein and R. Shakarchi. *Fourier Analysis: An Introduction*. Princeton University Press, 2003.

[9] G. R. Lee, R. Gommers, F. Waselewski, K. Wohlfahrt, A. O'Leary. "PyWavelets: A Python package for wavelet analysis." *Journal of Open Source Software*, 4(36):1237, 2019.

[10] P. Virtanen et al. "SciPy 1.0: fundamental algorithms for scientific computing in Python." *Nature Methods*, 17:261–272, 2020.